Kellen Jones
CS 370 Systems and Architecture
Pallavi Deogade
Oct 13, 2026

CS 370 Midterm Report: Virtual Character Device Driver

1. Project title and purpose

Linux Driver Development Exploration: A Virtual Character Device Driver. The project builds a loadable Linux kernel module that registers a virtual character device (/dev/mychardev), plus a user-space C program that talks to it through system calls.

Linux distributions often lack manufacturer-provided drivers, which can be frustrating. I chose this topic to learn how drivers are created, so I can eventually write my own when an accessory does not work. A driver sits at the boundary between user programs and the kernel, which makes it a natural fit for CS 370. Because kernel development is new to me, the scope is a virtual device rather than real hardware, as approved by the instructor.

2. Project goals

- The goal is a small driver that I can fully build, test, and explain.
- Build a loadable Linux kernel module that registers a virtual character device.
- Write a user-space C program that communicates with the device through open, read, write, and close.
- Understand how data moves safely between user space and kernel space.
- Analyze the compiled module with readelf, nm, and objdump, and connect the results to CS 370 concepts.
- Document the process, including problems and fixes, so the approach can be reused for future drivers.

3. What I have implemented

- **Environment:** Ubuntu 26.04.1 LTS on a 2015 Intel MacBook (kernel 7.0.0-34-generic). The toolchain (gcc, make, gdb, lldb, strace, binutils, kernel headers) is installed and verified. Modules are loaded only in a QEMU/KVM Ubuntu 26.04.1 guest.
- **Stage 1, hello-world module:** mychardev.c has an init and an exit function that log with pr_info, plus MODULE_* metadata. It builds into mychardev.ko with a kbuild Makefile. Build output and the modinfo, nm, readelf, and objdump results are saved as evidence (section 10).
- **Stage 3, character device (built and loaded):** the init function allocates a device number, registers a cdev, and creates a class and device so /dev/mychardev

appears. The open, release, read, and write handlers log to dmesg. I built it on kernel 7.0 and loaded it in a QEMU/KVM guest: it allocated major 239, minor 0 and created the device node. For now read returns 0 (end of file) and write reports success but discards the data

- **Repository:** the project is in Git with a README, setup guide, dev log, and a saved evidence folder.

4. CS 370 concepts use and how they are reflected in my code and system

Concept	Where it appears	Evidence
Compilation	make invokes the kernel build system, which runs gcc and links the module	stage1_build.txt
Object files	mychardev.ko is a relocatable ELF object; calls to kernel functions are resolved when the module loads	nm, readelf output
Number systems and bitwise ops	MAJOR(dev_num) and MINOR(dev_num) are logged in mychardev_init; objdump and nm print hex addresses	major 239, minor 0 in dmesg; ls -l shows crw------- (octal 600)
Control flow	The if (ret < 0) checks and the goto cleanup ladder in mychardev_initcompile to compare-and-jump instructions	objdump -d of the init function
Registers and stack	The test program's calls to open pass the path in rdi and flags in rsi; stack frames show in lldb	lldb session (planned)
System calls	A file_operations table connects each system call on the device to a handler function	dmesg lines with open, write, read, and release. Open by pid 1590 (cat) Read of 65536 bytes requested Release by pid 1590 (cat)
Processes	open and release log current->pid and the process name	dmesg lines naming pid 1581 (tee) and pid 1590 (cat)
User vs. kernel space	read and write take user-space pointers; stage 4 will use copy_to_userand copy_from_user for them	**stage 4**

NOTE: THE LLDB AND STRACE ROWS ARE PLANNED AND NOT YET COLLECTED.

5. Testing performed and results

I built the module on kernel 7.0, loaded it in a virtual machine, and exercised every handler from the shell with echo, tee, and cat. The C test program is not extended yet.

Test	Result
Stage 1 module builds	Pass
Stage 3 module builds on kernel 7.0	Pass (only environment warnings)
Load module	Pass: Major 239, minor 0
Device node appears	Pass: crw——— root, 239, 0
Write	Pass: handler received 6 bytes (“hello” plus a newline)
Read	Pass: cat requested 65536 bytes and got end of file
Open and release logs	Pass: lines name the pid and command
Unload	Pass: “module unloaded” logged
Edge cases (empty or oversized buffer)	Not yet: needs the stage 4 buffer

Figure 1.

```
[   53.264056] mychardev: allocated major 239, minor 0
[   53.264329] mychardev: module loaded, /dev/mychardev created
[  222.023502] mychardev: open by pid 1581 (tee)
[  222.023724] mychardev: write of 6 bytes received
[  222.024040] mychardev: release by pid 1581 (tee)
[  222.085122] mychardev: open by pid 1590 (cat)
[  222.085209] mychardev: read of 65536 bytes requested
[  222.085327] mychardev: release by pid 1590 (cat)
[  222.140615] mychardev: module unloaded
```

dmsg output

6. Challenges encountered and how I addressed them

- **Needing an x86 environment:** I work on Apple silicon, so I put Ubuntu on an older Intel Mac and now work on it over SSH.

- **Test VM permissions:** virsh failed with a permission error on the libvirt socket. The cause was that group membership only applies to new logins, reconnecting fixed it.
- **The cloud image VM rejected my login at first:** I learned cloud-init reads its config only on first boot, and after correcting the password I could log in.
- **Kernel version differences:** Many tutorials target older kernels. I wrote the stage 3 code for the modern form and test-compiled it against 6.8 headers and it built cleanly on 7.0.
- **Protecting personal details in a public portfolio repo:** Build output contains usernames and paths, so I redact saved evidence and review it before committing.
- **Keeping the host safe:** A buggy module can crash the machine, so I only build and analyze on the host, and loaded modules only inside the VM.
- **Unsigned module taints the kernel:** Loading logged “module verification failed” and “taints kernel.” This is expected for a self-built module. It works because Secure Boot is off, and signing would be needed otherwise.
- **VM kernel changed under me:** The guest updated from 7.0.0-34 to 7.0.0-38, so the module has to be built against the running kernel’s headers.
-

7. What remains to be developed

- The core of the driver, the data path between user space and kernel space, is still ahead.
- Add the fixed-size kernel buffer with copy_from_user in write and copy_to_user in read (stage 4).
- Extend the user-space test program to write a string, read it back, and print buffer addresses.
- Test edge cases: empty reads, writes larger than the buffer, and opening the device when the module is not loaded.
- Collect strace and lldb evidence for the system calls, registers, and stack.
- Stretch goals, only if time permits: a mutex around the buffer, an ioctl command, and an open counter.

8. CS 370 concepts I plan to apply next

Without knowing exactly what we will be covering, Likely candidates:

- Pointers and memory layout: comparing the addresses of a user buffer and the kernel buffer, and explaining why they differ.
- The stack and function calls in more depth: examining stack frames and return addresses in the test program with lldb.
- Linking and symbols: how undefined symbols in the .ko are resolved when the module loads.

- Processes: which process opens the device, observed through dmesg and strace.
- Memory hierarchy and caching: if covered, how buffer size and access patterns affect copying.

9. Technical evidence

Tool	Result
Build Output	CC [M], MODPOST, and LD [M] produce mychardev.ko
nm	Defines init_module, mychardev_open/read/write, and mychardev_fops; leaves cdev_add, device_create, and _printk undefined (U) until load time
readelf -h	Type REL (relocatable), x86-64, entry point 0x0, no program headers
readelf -S	51 sections (.text, .init.text, .rodata, .bss, .modinfo); all addresses 0 until load; .rela.* sections hold relocations
objdump -d	Excerpts below: argument registers, a compare-and-jump, and a shift and mask
dmsg	Major 239, minor 0; open, write, read, and release lines naming the pid and command

```
; registers: arguments to alloc_chrdev_region
16: xor  %esi,%esi          ; arg 2 = 0
1f: mov  $0x1,%edx          ; arg 3 = 1
24: mov  $0x0,%rdi          ; arg 1 = &dev_num (filled in at load
time)

; control flow: if (ret < 0)
34: test %eax,%eax
36: jns  4b                 ; skip the error path when ret ⩾ 0

; bitwise: MAJOR() and MINOR()
5a: shr  $0x14,%esi         ; major = dev_num >> 20
5d: and  $0xfffff,%edx      ; minor = dev_num & 0xFFFFF
```

The first four arguments travel in rdi, rsi, rdx, and rcx; if (ret < 0) becomes test plus a conditional jump; and MAJOR/MINOR compile to a shift and a mask.